# Flutter UI ve Widget Mimarisi




Abdussamed Hırabaşoğlu - 2024481009
Ömer Murat - 2024481021
Esma Taş - 2024481027

# Flutter Nedir? Tek Kodla Her Platform

Tanım · Mimari

## FLUTTER NEDİR? KISACA TANIM

Flutter, Google'ın geliştirdiği açık kaynaklı bir arayüz (UI) geliştirme kitidir. Dart diliyle yazdığınız tek bir kod tabanından iOS, Android, web, Windows, macOS ve Linux uygulamaları üretirsiniz.

| Google | Dart | 2018 |
| --- | --- | --- |
| Geliştiren ve destekleyen | Kullanılan programlama dili | Kullanıma sunulduğu yıl |

### Temel fikir: Her şey bir widget

Buton, metin, boşluk, hatta sayfa düzeni bile bir widget'tır. Küçük widget'ları iç içe koyarak ekranın tamamını (widget ağacı) oluşturursunuz.

## FLUTTER NASIL ÇALIŞIR? 3 KATMANLI MİMARİ

1 **Framework · Dart**

Widget'lar, animasyon, Material ve Cupertino: arayüzü yazdığınız katman

2 **Engine · Çizim motoru**

Skia / Impeller ile pikselleri doğrudan ekrana çizer

3 **Platform · Embedder**

Android, iOS, Web, Windows, macOS ve Linux üzerinde çalıştırır

Platformun hazır bileşenlerini değil kendi çizim motorunu kullanır; bu yüzden her cihazda aynı görünüm sağlar.

# Flutter Ne İşe Yarar?



Flutter’ın asıl amacı, tek bir kod tabanıyla farklı platformlarda hızlı, akıcı ve tutarlı kullanıcı arayüzleri geliştirmektir. Aşağıdaki dört amaç için tercih edilir.

### Amaç 1: Tek kodla çok platform

İki ayrı uygulama yerine tek Dart kodu yazılır; Android, iOS, web ve masaüstü için ortak kullanılır.

### Amaç 2: Hızlı geliştirme ve test

Hot reload ile değişiklik yeniden başlatmadan ekranda görülür; prototip ve hata düzeltme hızlanır.

### Amaç 3: Tutarlı ve özgün arayüz

Kendi çizim motoru sayesinde tasarım her cihazda aynı görünür; özel, animasyonlu ekranlar kurulur.

### Amaç 4: Maliyet ve zaman tasarrufu

Tek ekip ve tek kod bakımıyla ürün pazara daha kısa sürede ve daha düşük maliyetle çıkar.



Android | iOS | Web | Windows | macOS | Linux

*Tek kodla hedeflenen platformlar*

# Flutter Nerelerde Kullanılır?



## PLATFORMLAR

### Mobil

Android ve iOS için tek kodla mobil uygulamalar.

### Web

Tarayıcıda çalışan web uygulamaları ve arayüzleri.

### Masaüstü

Windows, macOS ve Linux için masaüstü uygulamaları.

## YAZILIMSAL KULLANIM ALANLARI

### Gömülü sistem ve IoT

Embedder ile Linux tabanlı akıllı ekran, araç içi bilgi-eğlence paneli ve kiosk arayüzleri

### Bulut ve API istemcisi

REST, GraphQL, gRPC ve Firebase ile arka uca bağlanan, gerçek zamanlı veri akışlı istemciler

### Veri görselleştirme

CustomPaint ile özel grafikler, canlı izleme panelleri ve yönetim (dashboard) arayüzleri

### Oyun ve 2D etkileşim

Flame oyun motoru, ticker tabanlı animasyon ve yüksek kare hızlı özel arayüzler

# Öne Çıkan Özellikler



### Hot Reload

Kod değişikliklerini uygulamayı yeniden başlatmadan anında ekranda görürsünüz.

### Widget tabanlı arayüz

Her şey bir widget'tır; küçük parçalar birleştirilerek karmaşık ekranlar kurulur.

### Yüksek performans

Kendi çizim motoruyla native'e yakın, akıcı bir kullanıcı deneyimi sunar.

### Zengin widget kütüphanesi

Material Design ve Cupertino (iOS) bileşenleri hazır olarak gelir.

### Dart dili

İstemci tarafı geliştirme için optimize edilmiş, kolay öğrenilen nesne yönelimli dil.

### Açık kaynak ve topluluk

Google desteği, büyüyen topluluk ve geniş paket ekosistemi.

**Bilinmesi gerekenler:** kamera, bildirim ve ödeme gibi platforma özgü özellikler için ek ayar veya eklenti gerekebilir; uygulama boyutu native'e göre daha büyük olabilir.



# İçindekiler

Flutter UI ve Widget Mimarisi

## Temeller


01 Giriş

02 Temel Widget’lar

03 Material Design

04 Cupertino (iOS)


## İçerik ve Etkileşim


05 Metin

06 Kullanıcı Girdisi

07 Etkileşim

08 Düzen


## Kaydırma ve Görsellik


09 Kaydırma

10 Sliver Mimarisi

11 Varlıklar ve İkonlar

12 Stil ve Tema

13 Çizim ve Efektler


## Hareket ve Erişim


14 Animasyon

15 Asenkron İşlemler

16 Erişilebilirlik

01

# Flutter UI ve Widget Mimarisine Giriş

Introduction

TEMEL FELSEFE

## “Her şey bir widget’tır”

Yapısal elemanlar, stil özellikleri, düzen şemaları ve hatta etkileşimlerin kendisi birer widget olarak tanımlanır. Flutter bu widget’ları bir ağaç (tree) yapısında organize eder ve ekrandaki pikselleri doğrudan çizer.

**Performanslı rendering**

Piksellerin doğrudan çizilmesi hız kazandırır

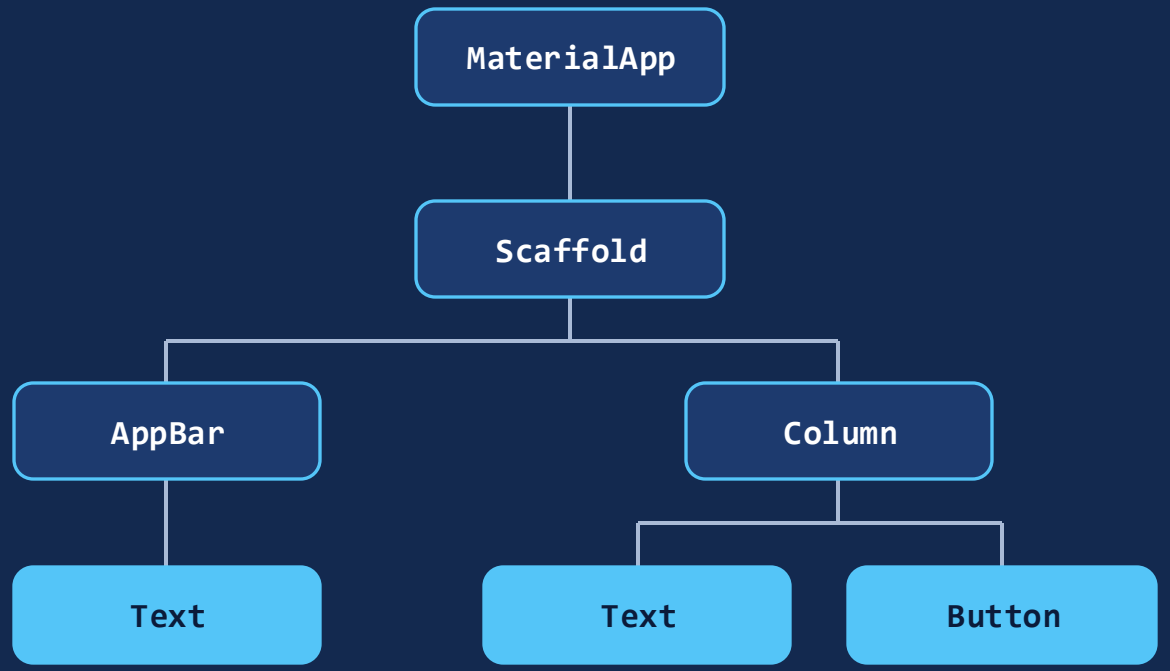
**Küçük, bağımsız parçalar**

Karmaşık arayüzler parça parça inşa edilir

**Yeniden kullanılabilirlik**

Aynı widget birden fazla yerde kullanılır

WIDGET AĞACI (WIDGET TREE)




Her kutu bir widget’tır. Flutter ağacı yukarıdan aşağı dolaşır ve ekrandaki pikselleri doğrudan çizer.

# 02 Temel Widget’lar

Basics




Uygulama arayüzünün iskeletini oluşturan, en sık kullanılan yapı taşlarıdır.

### Scaffold

Uygulamanın temel sayfa yapısını kurar; üst bar (app bar), alt gezinme menüsü (bottom navigation) ve yan menü (drawer) için hazır alanlar sunar.

### AppBar

Ekranın üst kısmında yer alır; başlığı ve aksiyon butonlarını barındırır.

### Container

Diğer widget’ları içine alan çok amaçlı bir kutudur; kenar boşluğu (margin), dolgu (padding), renk, kenarlık ve boyut gibi stil özellikleri kazandırır.

# 03 Material Design Widget'ları

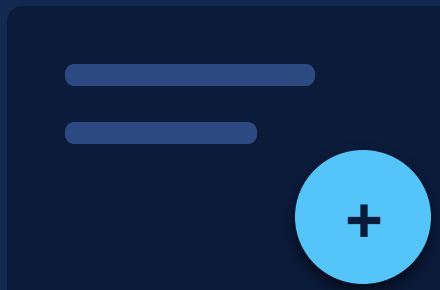
Material

Google'ın Material Design sistemini birebir uygulayan, Android'in alışılmış görünümünü sunan kapsamlı bir kütüphanedir. Sadece Android'de değil, tüm platformlarda tutarlı bir tasarım dili sağlar.



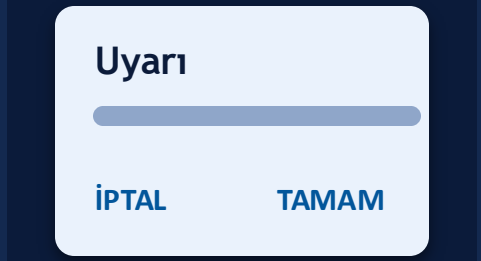
## Card

Gölgeli yüzeyler sunar; ilgili içeriği yükseltilmiş bir kart içinde gruplandırır.



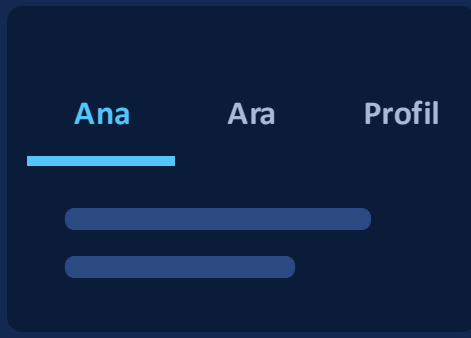
## FloatingActionButton

Ekranın köşesinde havada duran eylem butonudur; ana eylemi öne çıkarır.




## AlertDialog

Kullanıcıya uyarılar gösterir ve onay ister.

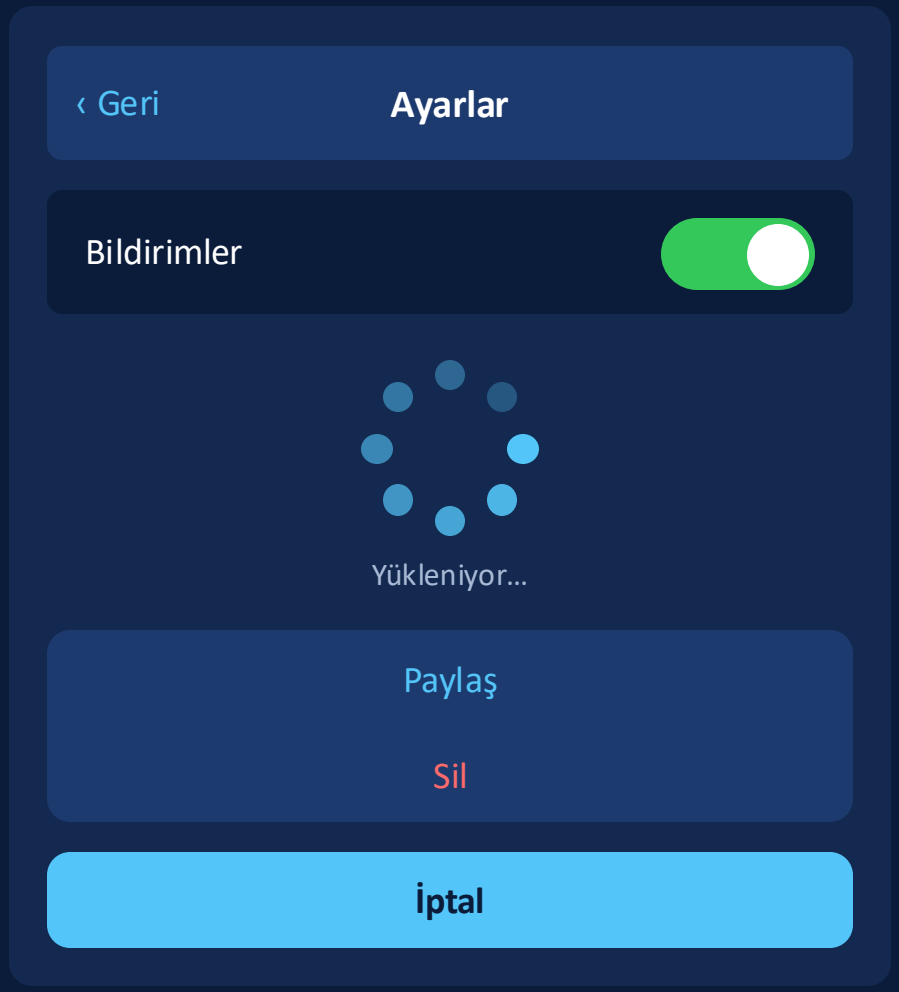



## TabBar

Çok sekmeli sayfa geçişlerini sağlar.

# 04 Cupertino (iOS) Widget'ları

Cupertino




Apple'ın "Human Interface Guidelines" standartlarına uygun, iOS'un doğal görünüm ve hissiyatına sahip arayüz elemanlarıdır. Uygulama iOS cihazlarda çalışırken kullanıcıya daha tanıdık bir deneyim sunar.

**`CupertinoNavigationBar`**

iOS tarzı üst menü; sayfa başlığı ve geri gezinme alanı.

**`CupertinoSwitch`**

iOS tarzı aç-kapat anahtarı.

**`CupertinoActionSheet`**

Alttan açılan eylem menüsü.

**`CupertinoActivityIndicator`**

iOS tarzı yükleme animasyonu.

# 05 Metin Kullanımı ve Yönetimi

Text

ÖNİZLEME

Text

**Merhaba Flutter**

RichText + TextSpan

Bir **kalın** ve altı çizili kelime

DefaultTextStyle

*Başlık metni*
*Açıklama metni*
*Not metni*

Ekranda yazı göstermek, şekillendirmek ve hizalamak için kullanılan bileşenlerdir.

### Text + TextStyle

Tek tip stile sahip standart metinler oluşturur; TextStyle ile renk, boyut, kalınlık ve font ailesi belirlenir.

### RichText + TextSpan

Aynı paragraf içinde farklı renk veya kalınlığa sahip (örn. bir kelime kalın, diğeri altı çizili) metinler oluşturur.

### DefaultTextStyle

Kendi altındaki tüm metin widget'larına varsayılan bir stil uygulayarak kod tekrarını önler.

06

# Kullanıcı Girdisi Alma

Input

Kullanıcılardan bilgi toplamak ve form işlemleri yapmak için kullanılan veri giriş araçlarıdır.

### TextField / TextFormField

Metin girişi için TextField; form doğrulaması (validation) desteği sunan TextFormField kullanılır.

### Checkbox / Radio

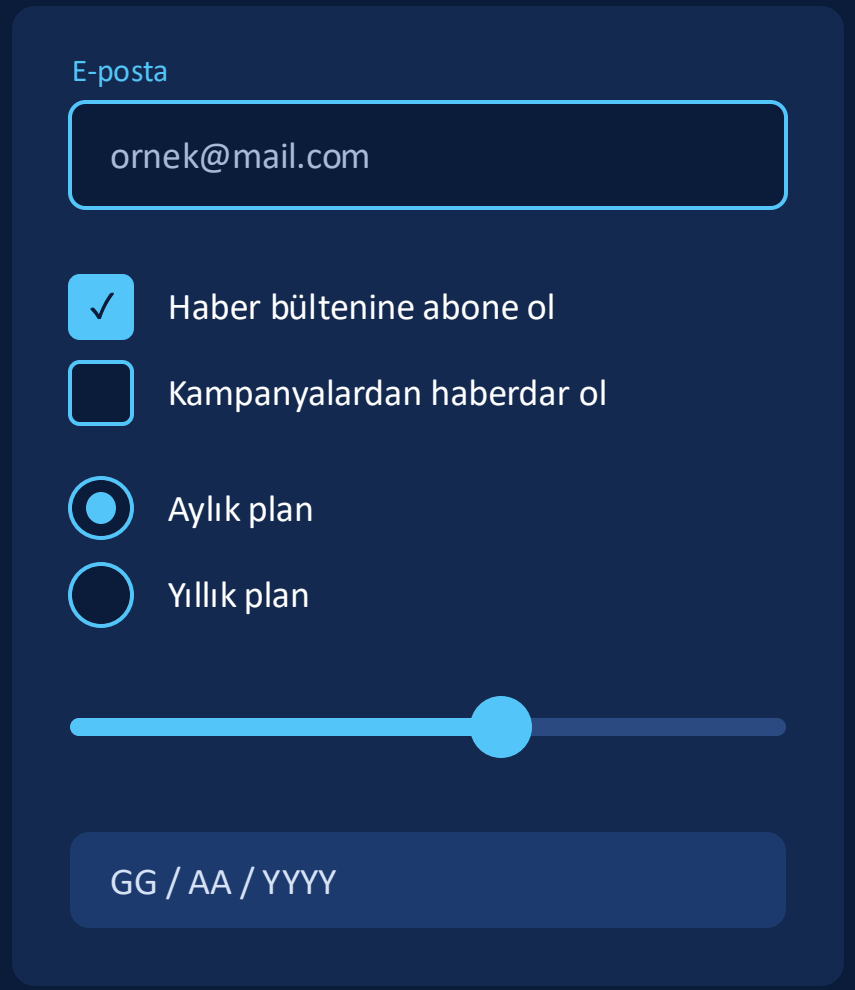
Kullanıcının seçenekler arasından tercih yapmasını sağlar: Checkbox çoklu, Radio tekli seçim sunar.

### Slider

Belirli bir değer aralığında ayarlama yapmaya yarar.

### Picker

Tarih ve saat seçimi için kullanılan çeşitli “Picker” bileşenleridir.



# 07 Kullanıcı Etkileşimi

Interaction

Kullanıcının ekrana dokunma, çift tıklama, uzun basma, kaydırma veya sürükleme gibi fiziksel jestlerini algılayan yapısal bileşenlerdir.

## ALGILANAN JESTLER

- **Dokunma**
- **Çift tıklama**
- **Uzun basma**
- **Kaydırma**
- **Sürükleme**

### `GestureDetector`

Görsel bir çıktısı yoktur; içine konulan widget'a her türlü dokunmatik etkileşim yeteneği kazandıran temel sınıftır.

### `InkWell`

Material Design'ın dalgalanma (ripple) efektiyle dokunma tepkisi vermek için tercih edilir.

### `Draggable + DragTarget`

Ekranda sürükle-bırak işlemleri için kullanılır.




### `Dismissible`

Liste elemanlarını yana kaydırarak silmeyi sağlar.

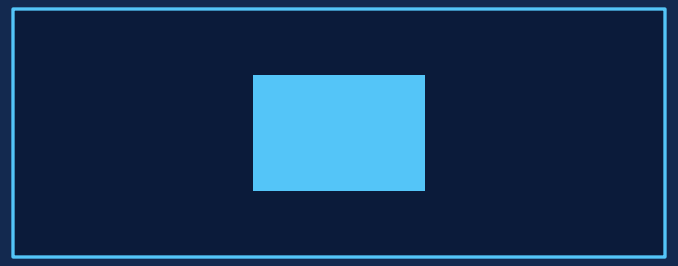


# 08 Düzen (Layout) Widget'ları

Layout

Widget'ların ekranda nasıl konumlanacağını, hizalanacağını ve boyutlandırılacağını belirleyen taşıyıcı bileşenlerdir.

## TEK ÇOCUKLU (SINGLE-CHILD)



### Center

İçerikleri merkeze alır

### Padding

Etrafında boşluk bırakır

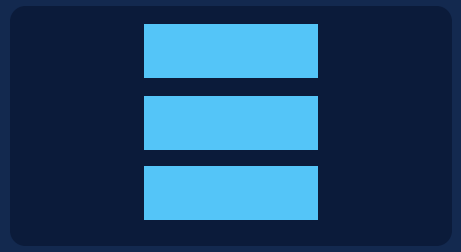
### Align

Hizalama yapar

## ÇOK ÇOCUKLU (MULTI-CHILD)



**Row**
yatayda sıralar



**Column**
dikeyde sıralar



**Stack**
Z ekseninde üst üste bindirir

### Expanded / Flexible

Mevcut boş alanın çocuklar arasında nasıl paylaştırılacağını kontrol eder.

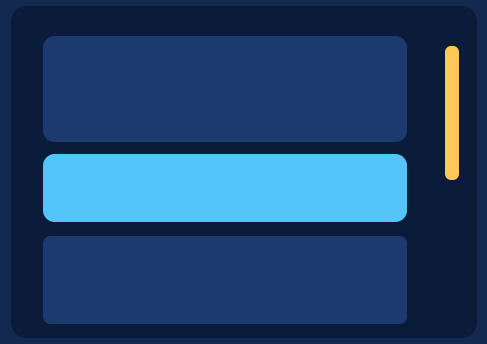


09

# Kaydırılabilir Alanlar

Scrolling

İçeriğin cihaz ekranına sığmadığı durumlarda kaydırma özelliği kazandıran bileşenlerdir.



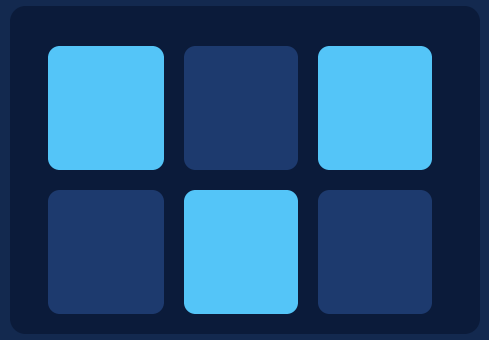
### SingleChildScrollView

Basit ve tek bir parça içeriği kaydırılabilir yapmak için kullanılır.



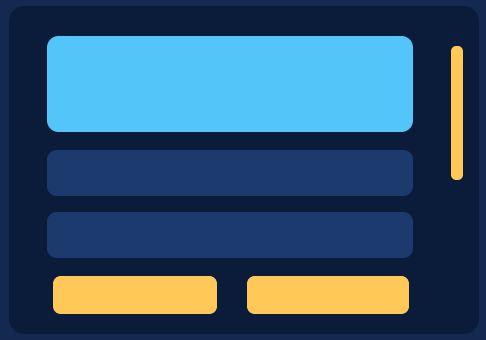
### ListView

Aynı türden çok sayıda elemanı dikey veya yatay bir listede gösterir.



### GridView

Elemanları ızgara görünümünde, satır ve sütunlar halinde gösterir.



### CustomScrollView

Kaydıkça küçülen üst bar ve farklı liste yapılarından oluşan gelişmiş ekranlar için “Sliver” bileşenleriyle kullanılır.

**Basit içerik → Çok elemanlı liste → Izgara → Gelişmiş ekran kompozisyonu**

# 10 Sliver Mimarisi

Sliver

CustomScrollView, farklı kaydırma davranışlarını tek bir akıcı ekranda birleştirir; her parça bir “sliver”dır.




### SliverAppBar

Kaydırdıkça daralan, dinamik kapak başlığı oluşturur.

### SliverList + SliverGrid

Hibrit ekranlar: aynı kaydırma alanında hem liste hem ızgara kullanılır.

### Bellek dostu

Yalnızca ekranda görünen dilimleri (sliver) çizer; uzun listelerde performans korunur.

# 11 Varlıklar, Görüntüler ve İkonlar

Assets

pubspec.yaml

```
flutter:
  assets:
    - assets/images/logo.png
    - assets/icons/

Image.asset('assets/images/logo.png')
```

## Image

Ağdan, yerel dosya sisteminden, uygulama klasöründen (asset) veya bellekten resim yükler.

- `Image.network` Ağ
- `Image.file` Dosya sistemi
- `Image.asset` Uygulama klasörü
- `Image.memory` Bellek

## pubspec.yaml

Tüm görsel varlıkların projeye dahil edilmesi bu dosya üzerinden deklare edilerek yönetilir.

## Icon

Yüksek çözünürlüklü ve ölçeklenebilir ikonlar ekler; Material/Cupertino ikon setleri tercih edilir.

# 12 Stiller ve Temalandırma

Styling

Uygulamanın genel görsel tutarlılığını sağlamak için kullanılan renk, tipografi ve boyutlandırma araçlarıdır.

## Theme / ThemeData

Ana renkleri (primary color), arka plan renklerini ve varsayılan font stillerini tek bir merkezden tanımlar ve tüm widget ağacına uygular.




*Tüm widget ağacı aynı temayı kullanır*

## MediaQuery

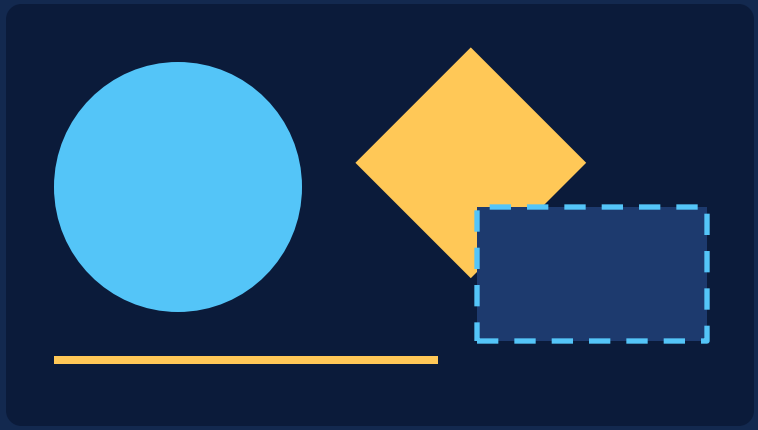
Cihazın çevresel özelliklerine erişerek arayüzü dinamik olarak şekillendirir: ekran boyutu, çözünürlük, cihaz yönü ve aydınlık/karanlık mod tercihi.



# 13 Çizim ve Görsel Efektler

Painting

Standart widget'ların sınırlarını aşarak arayüze özel grafikler, şekiller ve görsel manipülasyonlar eklemeyi sağlayan araçlardır.



### CustomPaint / CustomPainter

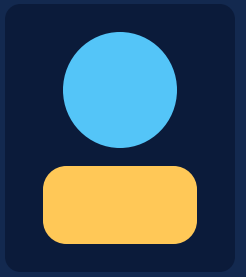
Tamamen özelleştirilmiş, kanvas (canvas) tabanlı 2D çizimler yapmak için kullanılır.

### Transform



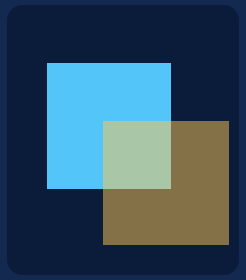
Bir bileşenin döndürülmesini, ölçeklendirilmesini veya yamultulmasını sağlar.

### ClipRRect / ClipOval



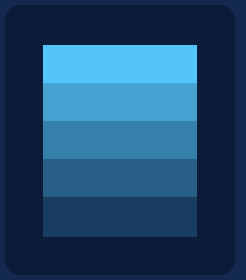
Bileşenlerin kenarlarını keser veya yuvarlatır.

### Opacity



Bir nesnenin şeffaflık seviyesini ayarlar.

### DecoratedBox



Arka plana karmaşık gradyan veya gölge efektleri verir.

# 14 Animasyon ve Hareket

Animation

Kullanıcı deneyimini canlandıran ve arayüzü daha akıcı hissettiren hareket bileşenleridir. Flutter iki temel animasyon yaklaşımı sunar.

## Örtük (Implicit)

Boyut, renk veya konum değişikliklerini otomatik olarak yumuşak bir geçişle sağlar.




`AnimatedContainer` `AnimatedOpacity`

## Açık (Explicit)

Geliştiricinin zamanlamayı, tekrarı ve hızı tam olarak kontrol ettiği animasyonlardır.




`AnimationController` `Tween`

**Sayfa geçişlerinden mikro etkileşimlere kadar geniş bir yelpazeyi kapsar.**

# 15 Asenkron İşlemler ve Arayüz

Async

İki builder da veri geldikçe arayüzü günceller; fark, verinin tek seferlik mi yoksa sürekli mi geldiğindedir.

## FutureBuilder · Tek seferlik akış

**1 İstek başlar**

API veya veritabanı sorgusu

**2 Bekleniyor**

Yükleme animasyonu gösterilir

**3 Tamamlandı**

Gelen veri ekrana çizilir

**4 Hata durumu**

Kullanıcıya hata mesajı sunulur

## StreamBuilder · Canlı akış

Sürekli veri akışını dinler; yeni veri geldikçe tüm sayfayı değil yalnızca ilgili bileşeni yeniler. Örnek: canlı sohbet, sensör verisi, borsa.



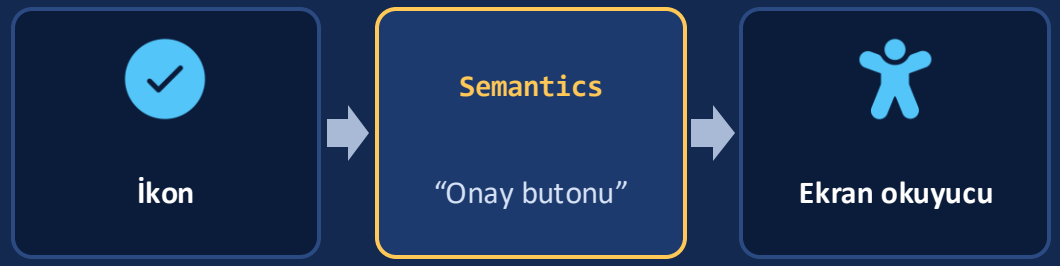
# 16 Erişilebilirlik

Accessibility

Uygulamaların, görme veya işitme engeli gibi kısıtlamaları olan kullanıcılar tarafından ekran okuyucular gibi yardımcı teknolojilerle rahatça kullanılabilmesini sağlayan özelliklerdir.

## Semantics

Arayüzdeki öğelerin ekranda nasıl göründüğünden ziyade “ne işe yaradığını” cihaza bildirir (örneğin bir ikonun sadece bir görsel değil, bir onay butonu olduğu bilgisi).




*Öğenin ne işe yaradığı cihaza bildirilir*

## ExcludeSemantics

Tamamen dekoratif öğelerin ekran okuyucular tarafından yok sayılmasını sağlar; engelli kullanıcılar için daha temiz ve odaklı bir deneyim oluşturur.




*Gereksiz gürültü ortadan kalkar*

# Genel Özet: Neler Öğrendik?

Summary

Flutter’da her arayüz, widget’lardan oluşan bir ağaçtır. Bu sunumda o ağacı kuran, düzenleyen, hareketlendiren ve herkes için erişilebilir kılan parçaları adım adım inceledik.

## 1 Temel fikir

Her şey bir widget’tır. Küçük, bağımsız parçalar bir ağaçta birleşir; Flutter bunları doğrudan ekrana çizer.

## 2 Arayüzü kurmak

Scaffold, AppBar ve Container iskeleti oluşturur. Material ve Cupertino hazır bileşenlerle Android ve iOS görünümü verir.

## 3 İçerik ve girdi

Text ve RichText yazıyı gösterir. TextField, Checkbox ve Slider kullanıcıdan veri alır.

## 4 Düzen ve etkileşim

Row, Column ve Stack yerleşimi belirler. ListView ve GridView kaydırır. GestureDetector ve InkWell dokunmaya yanıt verir.

## 5 Stil ve görsellik

ThemeData ve MediaQuery uyumu sağlar. Image, Icon, CustomPaint ve animasyonlar arayüze görsel zenginlik katar.

## 6 Veri ve erişilebilirlik

FutureBuilder ve StreamBuilder asenkron veriyi arayüze taşır. Semantics herkesin uygulamayı kullanabilmesini sağlar.



# Teşekkürler!

**Flutter ile her şey bir widget; sıra sizde: küçük bir parçayla başlayın, ağacı adım adım büyütün.**

## FLUTTER KURULUMU VE İLK ADIMLAR

1. **SDK'yı indirin**
   flutter.dev'den işletim sisteminize uygun paketi indirip çıkarın.
2. **Yolu ayarlayın**
   `flutter/bin` klasörünü PATH'e ekleyin; Android Studio veya VS Code kurun.
3. **Kurulumu doğrulayın**
   `flutter doctor` komutunu çalıştırıp eksikleri tamamlayın.
4. **Projeyi oluşturup çalıştırın**
   `flutter create my_app` ardından `flutter run`